CSA17 - Artificial Intelligence

Assignment 2 - CO2: Informed Search, Online Search, Adversarial Search, CSP and Optimisation

Field	Details
Student name	A HARITHA
Register number	192321147
Nature	Individual
Total marks	50
Submission date	_____

How the evidence was produced. Every count and time in this report was measured by running the Python code submitted with this report (experiments.py), which writes all raw numbers to results.json. Operation counts (nodes, checks, evaluations) are the primary evidence; execution times depend on the machine and are secondary. Each prediction is derived from the algorithm analysis in the relevant section and then compared with the measured result.

Assumptions. Grid moves are four-directional with unit cost. Random instances use fixed seeds, so every table can be reproduced by running experiments.py.

Task 1 - Greedy Best-First and A* Under Different Map Conditions (6 marks)

1.1 Predictions and their algorithmic basis

- **Uniform-cost search (UCS)** orders nodes by g alone, so it expands every cell with g below the optimal cost C*. On a 30 x 30 grid this is a diamond-shaped region; it is optimal but expands the most nodes on open maps.
- **Greedy best-first** orders nodes by h alone. On an open map it should head almost straight to the goal and expand few nodes, but it ignores path cost, so obstacles can make it return a longer path. In a cup-shaped trap placed on the straight line to the goal, it should expand many cells inside the cup before escaping.
- **A*** orders by f = g + h. With an admissible and consistent heuristic it is optimal, and it expands only nodes with f below C*. It should expand fewer nodes than UCS on the open and trap maps, and more than greedy on open maps.

1.2 Heuristic and its admissibility

h(n) = |row(n) - row(goal)| + |col(n) - col(goal)| (Manhattan distance). **Relaxed problem:** delete all obstacles. Every real path is then still a valid path in the relaxed grid with the same unit-cost moves, so the relaxed optimum equals the Manhattan distance, which is a lower bound on the true cost. Hence h(n) <= h*(n): admissible. Consistency: one move changes Manhattan distance by exactly 1 and costs 1, so h(n) <= c(n,n') + h(n'), which gives consistency and means A* never needs to reopen closed nodes.

1.3 Three designed maps

- **Open map (contrast 1):** 30 x 30, about 10% random obstacles, start (0,0), goal (29,29). Connected; checked by BFS.
- **Trap map (contrast 2):** 30 x 30 with an empty field and a cup-shaped wall on the straight line between start (15,0) and goal (15,29). The cup opens toward the start; its back wall faces the goal.
- **Serpentine maze (edge case):** 21 x 21 with horizontal walls and alternating gaps, giving one long corridor.

1.4 Measured results

Map	Algorithm	Expanded	Generated	Max frontier	Path cost	Optimal?
open	UCS	806	806	30	58	Yes
open	Greedy	64	132	69	62	No
open	A*	700	738	63	58	Yes
trap	UCS	875	875	30	45	Yes
trap	Greedy	174	233	60	45	Yes
trap	A*	485	545	75	45	Yes
maze	UCS	181	181	1	180	Yes
maze	Greedy	181	181	1	180	Yes
maze	A*	181	181	1	180	Yes

Straight-line (Manhattan) start-goal distance: open map 58, trap map 29, maze 36 (for the maze the optimal path is longer than this because of the walls).

1.5 Comparison of prediction with observation

- **UCS** behaves as predicted: it is optimal on all maps and expands the most nodes on the open map (806) and trap map (875).

- **Greedy** is cheap on the open map (64 expanded) but returned a suboptimal path there (cost 62 against optimal 58, about 6.9% longer). This confirms the prediction that greedy ignores path cost.
- **Trap map: prediction only partly held.** Greedy expanded 174 cells, well below A* (485) and UCS (875), and its path was optimal here. The trap was not deep enough to force a long detour, so the evidence does not show greedy being badly fooled. I did not make a larger trap to force that failure, because this assignment asks for the observed behaviour of the designed maps.
- **A*** was optimal on every map. Its expansions (700 on open, 485 on trap) lie between greedy and UCS, as predicted. It expanded more than greedy on the trap because it must expand all cells whose f is below C* to prove optimality.
- **Maze:** all three expand the same 181 cells, because the corridor is unique and the frontier never holds more than one node.

1.6 Design decision (95% open roads)

For the shuttle, route cost is what matters, so an optimal route is required. On the open map A* expanded 700 nodes against 64 for greedy, and it returns the optimal route. Greedy is about 10.9x cheaper but routes 6.9% longer on the open map, which is the case that occurs for 95% of trips. **Recommendation: A* with the Manhattan heuristic.** Its extra cost on open roads is modest (about 11x greedy in node expansions; A* wall-clock time was not separately measured), and it keeps optimality on detour maps. Greedy would only be defended if response time were the sole objective, and even then its routes are not guaranteed to be efficient.

Task 2 - When Is an Online Search Agent an Appropriate Engineering Choice? (4 marks)

Agent. Online depth-first search: the rover senses only its four neighbours, keeps a stack of visited cells, moves to an unvisited neighbour when one exists, and otherwise backtracks one step along the stack. Competitive ratio (CR) = moves travelled / optimal distance with full map knowledge (BFS). The LRTA* extension was optional and not implemented.

Size	Density p	Target	Runs	Found	Mean moves	Mean backtracks	Mean CR	Max CR
15x15	0.0	near	5	5	4	0	1.0	1.0
15x15	0.0	far	5	5	224	0	8.0	8.0
15x15	0.0	unreachable	5	0	436	218	-	-
15x15	0.2	near	5	5	4.8	0.4	1.2	2.0
15x15	0.2	far	5	5	81.6	5.4	2.91	3.71
15x15	0.2	unreachable	5	0	352.8	176.4	-	-
15x15	0.35	near	5	5	5.2	0.6	1.3	1.5
15x15	0.35	far	5	5	135.2	40.4	4.22	7.62
15x15	0.35	unreachable	5	0	66.4	33.2	-	-
30x30	0.0	near	5	5	4	0	1.0	1.0
30x30	0.0	far	5	5	870	0	15.0	15.0
30x30	0.0	unreachable	5	0	1786	893	-	-
30x30	0.2	near	5	5	4	0	1.0	1.0
30x30	0.2	far	5	5	372.6	101	6.38	12.39
30x30	0.2	unreachable	5	0	1420.8	710.4	-	-
30x30	0.35	near	4	4	3.8	0.2	1.17	1.67
30x30	0.35	far	4	4	143.5	44	3.07	6.96
30x30	0.35	unreachable	4	0	542.5	271.2	-	-

Unreachable targets. The agent always reports failure after exploring its whole component. Measured moves equal 2 x (component size - 1) in 29 of 29 runs, which is the expected cost of a full DFS tour (each cell entered once, then each tree edge walked back once). Every unreachable run therefore ended in a finite, verifiable failure.

Analysis of successes and failures. When the target is near, CR is close to 1 because DFS often walks straight there. When the target is far in an open 30 x 30 grid, DFS can follow a long corridor away from the target before turning back: the maximum CR in the table shows the penalty. Backtracks are a direct measure of wasted motion: in open areas they are close to the number of cells entered, and in dense mazes they grow with dead-end count.

Operating boundary. An online agent is a reasonable engineering choice when (a) the facility map is unknown and the rover must act immediately, (b) the environment is small enough that a full tour is affordable (in our runs, a 30 x 30 facility with 20% obstacles needs roughly 1,400 moves to exhaust the component), and (c) each facility is visited only a few times. I would stop recommending it when the facility is large, when targets are typically far apart (the CR grows with distance), when targets may be unreachable and must be declared absent quickly, or when the same facility is visited repeatedly. Repeated missions are the decisive factor: a learning agent such as LRTA* would reduce repeat costs, but an explicitly stored map removes the exploration cost after the first run.

Changed condition: 20 rovers across 20 facilities, maps stable for months

The original recommendation was built for unknown maps. With 20 fixed facilities that stay unchanged for months, the exploration cost is paid once per facility and then amortised over every subsequent mission. I therefore revise the recommendation: **explore each facility once with online DFS to build and store its map, then plan every mission with an optimal search (A* or BFS) on the stored map.** Online search remains the right tool for the first survey and for facilities whose layout changes. Re-surveying is triggered when a planned route is blocked, which keeps the stored map trustworthy. The competitive-ratio evidence supports this: online DFS can cost up to 15.0 times the optimal distance on a single far target, which is acceptable once but wasteful if repeated across months.

Task 3 - Engineer the Behaviour of Alpha-Beta Pruning (6 marks)

Test trees. All trees have branching factor b = 4 and depth d = 4, giving b^d = 256 leaves. Internal nodes are MAX at even depth and MIN at odd depth. Node counts include internal nodes (visited) and leaves (evaluated).

- **T1 - best move ordering (many cutoffs):** children are sorted best-first for the player to move at every node.
- **T2 - worst move ordering (few cutoffs):** children are sorted worst-first for the player to move.
- **T3 - additional condition, all leaves equal (ties):** every leaf has value 0.
- **T4 - unordered:** the same random leaves as T1 in their generated order (a realistic middle case).

3.1 Predictions (derived before examining the results)

- **T1:** with perfect ordering the minimal tree has b^ceil(d/2) + b^floor(d/2) - 1 = 4^2 + 4^2 - 1 = 31 leaves. Prediction: alpha-beta evaluates about 31 leaves.
- **T2:** with worst ordering almost no bound is set early, so cutoffs are rare. Prediction: close to the full 256 leaves.
- **T3:** a cutoff fires as soon as a >= b, which holds immediately on equal values. Prediction: ties behave like perfect ordering, so about 31 leaves, not 256. Minimax returns the same value (0) in every case.
- **T4:** between the extremes, with a prediction of a few tens to over a hundred leaves.

3.2 Measured results

Tree	Minimax value	Minimax visited	Alpha-beta leaves	Alpha-beta visited	Cutoffs	Same value?
T1 best order	31	341	31	62	21	Yes
T2 worst order	31	341	237	322	32	Yes
T3 equal leaves	0	341	31	62	21	Yes
T4 unordered	31	341	122	189	37	Yes

3.3 Comparison and explanation

- **Predictions held for T1 and T3 exactly** (31 leaves in both). T2 came in at 237 leaves, below 256 because ties within the worst ordering still allow a few cutoffs; the prediction of "close to full" was therefore correct in spirit. T4 gave 122 leaves, inside the predicted range.
- **Why T2 needs far more work than T1:** alpha-beta only prunes when a bound set in one branch proves that another branch cannot change the decision. With best ordering the first child of each node already sets a strong bound, so the other children are refuted quickly. With worst ordering each new child is better than the bound found so far, so nothing is cut.
- **What creates or prevents cutoffs:** placing the best move first at MAX nodes and the best (lowest) move first at MIN nodes. Equal-valued leaves help because a bound equal to the window edge is enough to cut.
- **What this reveals about evaluating only b and d:** the same tree shape gives between b^(d/2) and b^d leaves, a factor of 8 here (31 vs 256). The branching factor and depth fix the worst case, but the actual effort depends on move ordering, which is a property of the game and the heuristic, not of b and d. Any claim of complexity must state the ordering assumed.

Task 4 - Minimax and the Cost of Examining Every Possibility (6 marks)

Game. k x k n-in-a-row (win length k: rows, columns and both main diagonals). Terminal positions score +/-(1000 + remaining depth) so faster wins are preferred. **Evaluation function:** for each line that contains only X marks (or only O marks), add c^2 for c X marks (or subtract c^2 for O). A line with both players is ignored. The function is used only at the depth limit, and only when the position is not terminal.

4.1 Derivation of the node count

Let E be the number of empty cells at the root and d the depth limit. Each level multiplies the number of positions by one fewer empty cell, so the number of nodes at depth i is P(E,i) = E!/(E-i)!. Without pruning or early termination, the total node count is N(d) = sum over i from 0 to d of E!/(E-i)!. The leaves (evaluation calls) are the nodes at depth d. Early wins end a line before depth d, so measured counts can fall below this upper bound; for the 3 x 3 full game they do so substantially.

4.2 Designed settings and measurements

Board	Depth d	Empty E	Nodes (measured)	Bound N(d)	Evaluation calls	Time (s)
3x3	3	9	586	586	504	0.003
3x3	9	9	549946	986410	255168	1.477
4x4	2	16	257	257	240	0.003
4x4	3	16	3617	3617	3360	0.028
4x4	4	16	47297	47297	43680	0.401
5x5	2	25	626	626	600	0.006
5x5	3	25	14426	14426	13800	0.132

For every depth-limited search with no early wins the measured count equals the bound exactly (for example 3x3 d=3: 586; 4x4 d=4: 47,297; 5x5 d=3: 14,426). Only the 3x3 full game departs from the bound (549,946 measured against 986,410), because wins end lines early, which the bound does not account for. Per additional level, the count multiplies by roughly E - i, consistent with the factorial formula.

4.3 Move quality (20 games vs random player, AI alternately first and second)

Setting	Wins	Draws	Losses	Avg time per AI move (s)
3x3 full search	20	0	0	0.2515
4x4 depth 2	16	4	0	0.0008
4x4 depth 3	16	4	0	0.0089
5x5 depth 2	14	6	0	0.0029

All 80 games were won or drawn; no game was lost. Quality rises with search depth, but at these settings the random opponent is weak, so the win rate alone cannot separate evaluation quality from search depth.

4.4 Scalability inquiry

Acceptability criterion. A move must be chosen within 1 second on this environment, so the game remains responsive. **Where plain minimax stops being practical.** The 4x4 depth-4 search took 0.40 s and 5x5 depth 3 took 0.13 s, both within the limit. The per-node cost is about 8.5 microseconds here. 5x5 at depth 4 needs about 318026 nodes in total, which extrapolates to roughly 2.7 s; depth 5 would need about 6693626 nodes, roughly 57 s. Plain minimax therefore becomes impractical on 5x5 boards beyond depth 3, and on any board where the nodes-per-depth factor (E - i) is large. The full 3 x 3 game (1.48 s from an empty board) is already at the limit.

Where the inefficiency originates. The search enumerates every ordering of moves (E!/(E-i)! positions), so the cost is governed by the factorial. It also re-evaluates identical positions reached by different move orders

(transpositions), and it prunes nothing. **What a redesign must do:** avoid visiting branches that cannot change the decision (alpha-beta with good move ordering), reuse results for transpositions (a transposition table keyed by board state), and bound the effort per move (iterative deepening within the time limit). It must also keep the evaluation function honest, since a shallow search relies on it heavily. None of these were implemented in this task.

Task 5 - Derive a Constraint-Propagation Algorithm from a Principle (7 marks)

5.1 Derived propagation rule and procedure

Representation. Each exam is a variable with a current domain (set of slots). Each pair of exams that share a student is a binary "not-equal" constraint, stored as two directed arcs (x, y) and (y, x). Constraint satisfaction is checked by comparing values, so the constraint graph is kept as adjacency lists.

Support. For an arc (x, y), value v in D(x) is *supported* if some w in D(y) satisfies w != v. If no such w exists, then every value y can take equals v, so x = v is impossible in any solution and v is removed.

Which arcs to re-examine. When D(x) changes, the supports of its neighbours z that depend on x may have disappeared. Therefore every arc (z, x), for z a neighbour of x other than the one that caused the change, is queued again. This is the standard AC-3 argument.

Failure. If any domain becomes empty, no value of that variable is consistent with its neighbours, so the problem has no solution.

```
function AC3(domains, neighbours):
    queue <- all arcs (x, y) with y a neighbour of x
    while queue not empty:
        (x, y) <- pop(queue)
        removed <- false
        for v in domains[x]:                  # check constraint: each value needs support
            if no w in domains[y] with w != v:
                remove v from domains[x]; removed <- true
        if removed:
            if domains[x] is empty: return FAILURE          # wipe-out: no solution
            for z in neighbours[x] except y: push (z, x) onto queue
    return CONSISTENT

function BacktrackWithPropagation(domains, assignment):
    if all variables assigned: return assignment
    x <- unassigned variable with smallest domain (MRV)
    for v in domains[x]:
        copy <- domains with domains[x] = {v}
        if AC3(copy, neighbours) = CONSISTENT:
            result <- BacktrackWithPropagation(copy, assignment + {x: v})
            if result found: return result
    return FAILURE
```

5.2 Implemented test cases

Instance A1 (solved by propagation alone): three exams in a chain 0-1-2. Domains before propagation: D(0) = {0}, D(1) = {0,1}, D(2) = {0,1}. The table below shows the domains after AC-3 (no search).

Exam	Before	After AC-3
0	[0]	[0]
1	[0, 1]	[1]
2	[0, 1]	[0]

Consistent after 7 constraint checks, with every domain a singleton: the assignment 0,1,0 is the solution and no search was needed. Value 0 was removed from exam 1 because its only neighbour, exam 0, must take 0; value 1 was removed from exam 2 for the same reason once exam 1 became 1.

Instance A2 (failure detected by propagation alone): chain 0-1-2 with D(0) = {0}, D(1) = {0,1}, D(2) = {1}. After 4 checks, D(1) is emptied (value 0 is forced by exam 0, value 1 is forced by exam 2), a domain wipe-out; the problem is declared unsolvable with no search.

5.3 Comparison of plain backtracking and backtracking with propagation

Random timetables: exam conflict graphs with edge probability 0.35 and 4 slots, 8 instances per size, MRV variable ordering. The search is capped at 20,000 nodes; a capped run is counted as incomplete, so

averages over capped runs are lower bounds.

Exams n	Plain: nodes	Plain: checks	Plain: backtracks	Prop: nodes	Prop: checks	Prop: backtracks	Prop: wipe-outs	Plain capped / Prop capped
10	19.8	30.5	9.8	10.0	676.9	0.0	0.0	0 / 0
15	2658.5	6741.1	2644.1	60.8	11457.2	47.6	21.4	1 / 0
20	12613.5	34187.5	12597.2	54.0	20986.8	44.0	18.0	4 / 0
25	19211.2	53086.2	19201.6	170.5	98524.0	170.5	69.0	7 / 0

Analysis. Propagation reduces search nodes dramatically: from 19211 to 170 at n = 25 on average, and the plain search reached its cap in 7 of 8 instances at n = 25 while propagation reached it in none. Each propagation node costs more: checks per run grow from 677 to 98524 when n grows from 10 to 25, so propagation trades checks for nodes. Plain backtracking grows roughly with the number of exams in the search tree; propagation keeps the number of nodes small but its check count grows with the size of the constraint graph and the number of times each arc is requeued.

5.4 Correctness: why an unsupported value never belongs to a solution

Claim: if value v is removed from D(x) because every w in D(y) equals v, then no solution assigns v to x. Argument by invariant. Let S be any solution that agrees with the current domains (initially every solution is contained in the initial domains). Suppose S gives x the value v. S must give y some value w. Since S is a solution, w is in D(y) (the invariant) and w != v (the constraint). But D(y) contains only v, so w = v, a contradiction. Hence no solution uses v for x, and removing v keeps every solution inside the domains. Each removal preserves the invariant, so a non-empty domain set at the end includes every solution, and an empty domain proves that no solution exists. This is reasoning about the constraint, not about program output.

Task 6 - Exhaustive Search, Local Search and Combinatorial Growth (8 marks)

6.1 Exhaustive enumeration design

Representation. A tour is a permutation of cities starting at the depot 0, read as a sequence of visits; the return to the depot closes the cycle. **Generation.** Enumerate all permutations of the n - 1 non-depot cities. **Evaluation.** Sum the n edge lengths of each tour. **Retention.** Keep the best (length, tour) pair seen so far, replacing it only on a strict improvement.

6.2 Correctness check on a hand-verifiable instance

Depot (0,0) and cities (0,3), (4,3), (4,0): a 3 x 4 rectangle. There are 3! = 6 orderings of the three non-depot cities. Each tour and its reverse have the same length, so the six permutations give three distinct cycles. By hand, the perimeter tour 0-1-2-3-0 has length 3 + 4 + 3 + 4 = 14, and the two diagonal-containing cycles are longer (for example 0-1-3-2-0 = 3 + 5 + 3 + 5 = 16). The algorithm returned 14.0, matching the hand calculation, and listed all six permutations:

Tour (as enumerated)	Length
0-1-2-3-0	14.0
0-1-3-2-0	16.0
0-2-1-3-0	18.0
0-2-3-1-0	16.0
0-3-1-2-0	18.0
0-3-2-1-0	14.0

6.3 Growth prediction and validation

Prediction. With the depot fixed, there are (n - 1)! distinct orderings. Each tour is generated exactly once, so the number of candidate tours evaluated is (n - 1)!. Going from n to n + 1 multiplies the work by n. Half of these are reversals of the others, but the enumeration does not exploit this, so the count is (n - 1)! and not (n - 1)!/2.

n	Tours evaluated	Predicted (n-1)!	Match?	Time (s)
5	24	24	Yes	0.0
6	120	120	Yes	0.0001
7	720	720	Yes	0.0007
8	5040	5040	Yes	0.0051
9	40320	40320	Yes	0.0432
10	362880	362880	Yes	0.4642

The count matched the prediction at every size. The time grows about 11x from n = 9 to n = 10, close to the factor of 9 predicted by the evaluation count.

6.4 Local search: 2-opt hill climbing with random restarts

Design. Start from a random tour and repeatedly apply any 2-opt move (reverse a segment) that shortens the tour, until none applies (a local minimum). A run consists of 5 random restarts keeping the best tour. 20 runs were made per instance with different seeds, and the optimum from exhaustive search was used as the reference.

n	Exhaustive optimum	Runs optimal (of 20)	Mean excess %	Max excess %	Mean time per run (s)
8	302.76	20	0.0	0.0	8.4e-05
9	307.98	20	0.0	0.0	0.0001
10	293.16	16	0.711	3.553	0.00012

Local search reached the optimum in every run for n = 8 and n = 9, and in 16 of 20 runs for n = 10, with mean excess 0.71% and worst excess 3.55%. Each run took well under a millisecond, against 0.46 s for exhaustive search at n = 10.

6.5 Inquiry: why small increases in n are expensive, and why local search can fail

- **Solution space.** Going from n to n + 1 adds one city and multiplies the number of tours by n. At n = 10 there are 362,880 orders; at n = 14 there are 6.2 billion. A single extra city changes the search from manageable to infeasible.
- **Local minima.** 2-opt accepts only strictly shorter tours, so a tour can stop at a configuration where every single reversal makes it longer, even though a distant configuration is shorter. Random restarts reduce this risk, which is why the 10-city runs occasionally stopped at 3.55% above optimum.
- **Plateaus.** When many tours have nearly the same length, the search may move sideways without improving and make no progress. With strict-improvement moves and continuous distances, plateaus are rare but still cause slow progress.

6.6 Deployment decision

I select a response budget of **one second per plan**, so that a planner can respond interactively. Exhaustive search took 0.46 s at n = 10 and grows by about a factor of 9 per extra city, so n = 11 would take roughly 4.7 s and is not acceptable. **Decision:** deploy exhaustive search for n up to 10 (where it is exact and within budget), and switch to 2-opt local search with restarts for n from 11 upward. Local search gave an optimal or near-optimal tour in all tested runs, so the switch costs little in quality. Task 7 shows that exact search can be extended well beyond n = 10 with a different algorithm.

Task 7 - Diagnose, Redesign and Validate (13 marks)

Selected solution: Task 6 exhaustive tour enumeration. It was chosen because its evidence shows a clear, measured limit (the factorial growth in Section 6.3) and because the operation counts can be compared directly with a redesign on the same instances.

A. Diagnosing the computational bottleneck (2 marks)

- **Where effort goes:** all of it goes into generating and summing tours. Each tour of n cities costs n additions, and there are (n - 1)! tours, so the cost is about n! operations.
- **Which operation grows:** the number of complete tours examined. Measured counts equal (n - 1)! exactly (Section 6.3).
- **Repeated work:** many tours share a prefix. For example, every tour that starts 0-1-2 contains the same partial length (0 to 1 to 2). Exhaustive search recomputes that partial sum in all (n - 3)! tours that share it. The same prefix appears again and again in different positions, so the work is repeated, not just large.
- **Characteristic causing the limit:** the algorithm treats each complete tour as independent. It never uses the fact that the cost of the rest of a tour depends only on the set of unvisited cities and the current city, not on the order of the visited ones.

B. Design objective (2 marks)

The redesign must (1) return exactly the optimal tour length on every instance where exhaustive search is possible (match rate 100%), and (2) reduce the number of basic cost operations from (n - 1)! to a count that grows as $n^2\ 2^n$. The measurable target is the ratio of evaluations to operations. A secondary constraint is memory, which will increase.

C. Revised computational strategy (4 marks)

Idea (dynamic programming over subsets, Held-Karp form). Define cost[S, j] as the length of the shortest path that starts at the depot, visits exactly the cities in subset S (a set of non-depot cities), and ends at city j in S. The state is what matters for the future: the best way to finish a tour from (S, j) does not depend on the order of the visits inside S. This is the optimal-substructure property, and it is what makes the redesign correct, not a hope that it is.

```
cost[{j}, j] = d(depot, j)                                  for each j
cost[S, j]   = min over k in S, k != j of  cost[S - {j}, k] + d(k, j)
best tour    = min over j of  cost[all, j] + d(j, depot)
(record the argmin k at each step to rebuild the tour)
```

- **What is exploited:** shared prefixes. Each (S, j) state is computed once and reused by every tour that passes through it.
- **What is avoided:** recomputing prefix sums and enumerating orders that differ only inside a subset when the endpoint is the same.
- **Preprocessing and memory:** the table has (n - 1) x 2^(n - 1) entries. At n = 10 that is 2,304 states; at n = 20 it would be about 10 million, so memory becomes the new limit.
- **New trade-offs:** time drops from factorial to roughly $n^2 2^n$, but memory grows exponentially. The method is exact, so there is no quality loss. It is the better method on time from about n = 8 upward; for very small n the simpler factorial method is fast enough and needs no table.

D. Prediction before implementation (1 mark)

Prediction: the redesign returns the same optimum as exhaustive search on every instance. The operation count should grow about as $n^2\ 2^n$, so the ratio to exhaustive evaluations should widen rapidly with n, and the time gain should appear first in time per run, with memory the cost that increases.

E. Implementation and validation (2 marks)

n	Exhaustive evaluations	Held-Karp operations	Ratio (exh / HK)	Held-Karp states	Exhaustive time (s)	HK time (s)	Same optimum?
5	24	56	0.4	32	0.0	0.0001	Yes
6	120	170	0.7	80	0.0001	0.0001	Yes
7	720	492	1.5	192	0.0007	0.0004	Yes
8	5040	1358	3.7	448	0.0051	0.0004	Yes
9	40320	3600	11.2	1024	0.0432	0.001	Yes
10	362880	9234	39.3	2304	0.4642	0.0028	Yes
12	39916800	56342	708.5	11264	-	0.0163	-
14	6227020800	319514	19489.0	53248	-	0.1051	-

For n = 5 to 10 the Held-Karp optimum matched exhaustive search on every instance. Operations rose from 56 to 9,234 across this range while exhaustive evaluations rose from 24 to 362,880. The ratio reached about 39 at n = 10 and grows with n. For n = 12 and n = 14 exhaustive search is not run (its counts, 11! and 13!, are given for comparison only); the redesign solved n = 14 in 0.11 s.

Note on the evaluation count. Held-Karp counts inner transitions (one addition and comparison each), while exhaustive search counts complete tours. A tour costs n additions, so the operation-level comparison is in the same order of magnitude but not identical; the timings above are therefore the fairer comparison of practical speed.

F. Defending the final design (2 marks)

- **Was the prediction supported?** Yes for correctness (100% match on n = 5 to 10). For growth, the operation count is consistent with the $n^2 2^n$ order, while exhaustive evaluations follow (n - 1)!.
- **Magnitude of improvement:** 168 times faster at n = 10 by measured time (0.464 s down to 0.0028 s), with the gap growing with n.
- **New costs:** memory of (n - 1) x 2^(n - 1) states, and a more complex implementation that must record argmins to rebuild the tour.
- **When the original may still be preferable:** for very small n (about n = 5 or less) the factorial method is simple, has no memory cost, and is fast enough. It is also easier to verify by inspection, which matters for the hand-checkable instance in Section 6.2.
- **Limit of the redesign:** at larger n (roughly n = 20 and above) the memory of the table, not the time, becomes the bottleneck, and local search (Section 6.4) becomes the practical choice, accepting a small quality loss.

Submission Contents

- Report: this document (Tasks 1 to 7).
- Source code: experiments.py (all algorithms and experiments, Tasks 1 to 7).
- Raw data: results.json (every operation count and timing used in this report).

Viva note. Every number in this report can be reproduced by running experiments.py. The Held-Karp redesign in Task 7 is implemented inside experiments.py (function held_karp).